

Round 1 : Future-facing preference is introduced



"If a summary/tip box is added later, use a light-gray background for that kind of box."

Temp preference stored



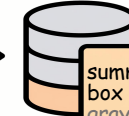
Delayed temp preference across turns

Round 1:
preference
mentioned



Future new
summary box
-> gray

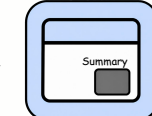
Stored in
working memory



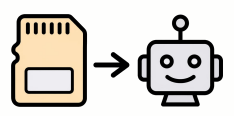
summary
box color:
gray

carried
across turns

Round N:
add summary box



Preference
injected



New Slide Results

Modify Round N : Read from working memory or not

New Slide Results

Attention and Positional Encoding: Capturing Dependencies and Injecting Order

- Scaled Dot-Product Attention: with Q, K, and V, $\text{softmax}(QK^T / \sqrt{d_k})$ gives the weights used to aggregate V.
- Multi-Head Attention: heads learn different subspace relations in parallel; the decoder uses both self-attention and cross-attention to the encoder.
- Positional encoding:
Added to input embeddings (same dimension d_{model}) at the bottom of the encoder and decoder.
Sinusoidal form: $\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{(2i/d_{\text{model}})})$
 $\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{(2i/d_{\text{model}})})$
Learned positional embeddings also work and perform similarly (Table 3, row E).

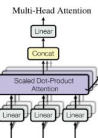
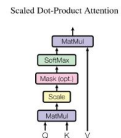


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

- Slide Summary**
- Scaled dot-product attention stabilizes optimization by using $QK^T / \sqrt{d_k}$ before softmax-weighted aggregation over V.
 - Sinusoidal positional encoding injects order and helps the model generalize to longer sequences.

! NO-Memory Injection
Relies on template/local context only

Injected memory : new box background use gray.



Memory



Prompt



Model



Model



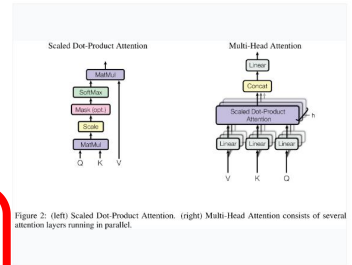
Prompt



No-Memory

Attention Mechanisms: Scaled Dot-Product and Multi-Head

- **Scaled Dot-Product Attention:** compute query-key dot products, scale by $\sqrt{d_k}$, apply softmax to obtain weights, and then aggregate values by weighted sum.
- **Multi-Head Attention:** learn different relationships in parallel across subspaces, then concatenate the heads and apply a final linear projection.
- Typical base setting (Table 3 example): $h=8$.



- Slide Summary**
- Scaled Dot-Product: $q \cdot k / \sqrt{d_k}$, softmax yields attention weights, and v is aggregated by weighted sum.
 - Multi-Head: h parallel heads learn in subspaces, then concatenate and linearly project; each head uses $d_k = d_{\text{model}} / h$.



Memory-Injection
Delayed memories carried over